

PROGRAMACIÓN APLICADA

Guía Formal del Estudiante

Desarrollo full-stack aplicado con proyecto integrador desde el primer día

Gestión académica 2026-2

Java 21 · Spring Boot · PostgreSQL · TypeScript · React · React Native · Docker · GitHub · CI · Spring AI
Documento de orientación académica, técnica y evaluativa para el estudiante

Propósito de esta guía

Este documento define cómo se aprenderá, cómo se trabajará en equipo, qué conocimientos deberá dominar cada estudiante y cómo se evaluará el avance del proyecto durante toda la asignatura. Debe ser utilizado como referencia permanente junto con las consignas específicas que entregue el docente.

1. Bienvenida y propósito de la asignatura

Programación Aplicada es la asignatura en la que el estudiante deja de resolver únicamente ejercicios de programación aislados y comienza a construir un sistema de software completo. El objetivo es aprender a transformar un problema real o verosímil en una solución que incluya datos persistentes, reglas de negocio, una API, una aplicación web, una aplicación móvil, pruebas, automatización de construcción y una integración controlada de inteligencia artificial.

La materia parte de los conocimientos previos de programación orientada a objetos en C++, pero introduce por primera vez varias áreas que deberán integrarse: diseño de bases de datos, PostgreSQL, Java 21, Spring Boot, arquitectura hexagonal, HTTP y REST, TypeScript, React, React Native, Docker, GitHub, integración continua y Spring AI. No se espera que el estudiante conozca estas tecnologías antes de iniciar; sí se espera disciplina para aprenderlas progresivamente y relacionarlas con el proyecto asignado.

Idea central

No se evaluará la capacidad de memorizar comandos o copiar tutoriales. Se evaluará la capacidad de comprender el sistema, justificar decisiones, construir funcionalidades, diagnosticar errores, colaborar profesionalmente y defender técnicamente el avance completo exigido a su equipo.

2. Qué deberá ser capaz de hacer al finalizar

- Descomponer un problema en actores, procesos, reglas de negocio, datos y funcionalidades verificables.
- Diseñar un modelo relacional básico y consistente, aplicando claves, cardinalidades, restricciones, normalización e integridad referencial.
- Escribir SQL de consulta y modificación con criterio, antes de depender de un ORM.
- Desarrollar un backend con Java 21 y Spring Boot, organizado como monolito modular con arquitectura hexagonal simplificada.
- Diseñar y consumir una API REST coherente, con validaciones, códigos HTTP y manejo de errores.
- Construir interfaces web con React + TypeScript y una aplicación móvil con React Native + TypeScript.
- Crear pruebas automatizadas útiles para lógica y flujos importantes.
- Utilizar Git y GitHub con ramas, issues/tareas, commits, pull requests y revisión entre pares.
- Levantar servicios mediante Docker y ejecutar un pipeline básico de integración continua.
- Integrar una capacidad concreta con Spring AI tratando la respuesta del modelo como no determinista y potencialmente fallable.
- Explicar end-to-end cómo una acción de usuario atraviesa interfaz, API, caso de uso, persistencia y respuesta.
- Defender técnicamente el proyecto completo, aunque cada integrante haya implementado diferentes partes durante el semestre.

3. El proyecto integrador: eje de toda la materia

Desde el inicio de clases, el docente conformará equipos de tres a cinco estudiantes y asignará a cada equipo un proyecto con alcance controlado. El proyecto no es una actividad adicional al curso: es el laboratorio permanente sobre el que se aplicará cada concepto enseñado. A medida que la asignatura avance, el mismo producto evolucionará desde la comprensión del problema y el modelo de datos hasta una solución web y móvil integrada.

Cada proyecto tendrá delimitaciones formales para los Trabajos Prácticos, el Primer Parcial, el Segundo Parcial y el Examen Final. La delimitación indicará qué capacidades, artefactos, decisiones técnicas y

funcionalidades deberán existir y ser defendibles en cada corte. Por tanto, el estudiante siempre sabrá cuál es la frontera evaluable vigente y no se le exigirá una parte futura del proyecto antes de la fecha definida.

Regla de alcance evaluable

Toda evaluación se realizará exclusivamente respecto al avance definido por el docente para ese proyecto y para ese corte. Sin embargo, dentro de ese avance, cualquier integrante puede ser preguntado sobre cualquier parte: datos, backend, frontend, móvil, pruebas, Docker, Git/CI, arquitectura o IA, según corresponda al momento de la materia.

4. Trabajo en equipo: condición de aprendizaje

El trabajo en equipo es obligatorio porque la materia también busca desarrollar competencias de Ingeniería de Software que no aparecen cuando una persona trabaja sola: coordinación, negociación de alcance, división de tareas, integración, revisión de código, resolución de conflictos, documentación de decisiones y responsabilidad compartida. Los equipos serán conformados por el docente con participación de los estudiantes para favorecer una distribución equilibrada de capacidades y compromiso.

Los integrantes pueden distribuirse tareas y especializarse temporalmente para avanzar con eficiencia. Sin embargo, una asignación de trabajo no crea propiedad exclusiva del conocimiento. Que un estudiante implemente una pantalla y otro una consulta SQL no significa que el primero pueda desconocer la base de datos ni que el segundo pueda desconocer la interfaz. Todo el equipo es responsable de comprender el estado integrado del producto dentro del alcance evaluable.

4.1 Roles y rotación

- Los roles representan responsabilidades temporales y no silos permanentes.
- El equipo debe rotar responsabilidades a lo largo del semestre para evitar que una sola persona concentre un área.
- Todo integrante debe realizar trabajo técnico verificable en el repositorio.
- Todo integrante debe revisar trabajo de otros compañeros mediante pull requests o revisión equivalente.
- Las decisiones importantes deben quedar documentadas y ser comprensibles por el equipo completo.
- La falta de conocimiento de una parte desarrollada por otro integrante no es una justificación válida durante una defensa individual.

4.2 Evidencia mínima de colaboración

- Issues, historias o tareas con responsable y criterio de aceptación.
- Commits pequeños y descriptivos vinculados al avance real.
- Ramas de trabajo y pull requests cuando el flujo del equipo lo requiera.
- Revisión de código por otro integrante antes de integrar cambios relevantes.
- Documentación técnica y funcional actualizada.
- Registro claro de decisiones, problemas encontrados y correcciones importantes.
- Repositorio integrado y demostrable, no archivos finales aislados enviados por mensajería.

5. Cómo se aprenderá: teoría, práctica y transferencia al proyecto

Cada tema tendrá una fundamentación teórica suficiente antes de convertirse en una receta de código. El estudiante deberá comprender el problema que resuelve cada concepto, su vocabulario, sus reglas esenciales, sus riesgos y su relación con el resto de la solución. Después se realizará una práctica guiada y,

finalmente, una transferencia al proyecto asignado. La materia busca que el estudiante aprenda a razonar sobre software, no sólo a reproducir sintaxis.

Una sesión típica de 90 minutos combinará activación conceptual, explicación y demostración, práctica guiada, aplicación al proyecto y cierre con evidencia. Algunas sesiones modificarán esta distribución para evaluaciones, laboratorios extensos o integración, pero la secuencia general será la misma: comprender, observar, hacer, integrar y explicar.

Momento	Propósito	Qué se espera del estudiante
0–10 min	Activación	Recordar conceptos previos, comprender el problema del día y relacionarlo con el proyecto.
10–35 min	Fundamentación + demostración	Construir el modelo mental y observar una implementación mínima razonada.
35–65 min	Laboratorio guiado	Aplicar el concepto con acompañamiento y aprender a diagnosticar errores.
65–82 min	Transferencia al proyecto	Incorporar una evidencia real al producto del equipo.
82–90 min	Cierre	Registrar evidencia, commit, dudas, decisiones y próximo paso.

6. Arquitectura objetivo del curso

El backend se construirá como un monolito modular con arquitectura hexagonal simplificada. Esto significa que la aplicación se despliega como una unidad, pero internamente se divide en módulos funcionales claros y en responsabilidades que separan dominio, aplicación e infraestructura. La finalidad pedagógica es aprender a controlar dependencias y evitar que controladores, persistencia y reglas de negocio queden mezclados.

No se utilizarán microservicios como arquitectura principal en esta materia. Los microservicios agregan problemas de despliegue distribuido, comunicación remota, observabilidad, consistencia y operación que distraerían de los fundamentos que primero deben dominarse. Aprender a diseñar bien un monolito modular es una base más sólida para comprender arquitecturas distribuidas posteriormente.

Zona	Responsabilidad	Ejemplos
Dominio	Reglas y conceptos esenciales del negocio sin depender de frameworks.	Entidades de dominio, value objects simples, reglas, excepciones de negocio.
Aplicación	Casos de uso y coordinación del comportamiento.	Servicios de aplicación, puertos de entrada/salida, DTOs de comando/consulta cuando corresponda.
Infraestructura	Tecnologías y adaptadores externos.	REST controllers, JPA repositories, PostgreSQL, configuración, clientes de IA.
Web	Interfaz navegador en React + TypeScript.	Componentes, formularios, estado, navegación, consumo de API.
Móvil	Interfaz móvil en React Native + TypeScript.	Pantallas, navegación, estado, consumo de la misma API.

7. Stack tecnológico de referencia

Área	Tecnología	Finalidad
Lenguaje backend	Java 21	POO, tipos, excepciones, colecciones, records y construcción del backend.
Framework backend	Spring Boot	Inyección de dependencias, REST, validación, persistencia, configuración y pruebas.
Base de datos	PostgreSQL	Modelo relacional, SQL, restricciones, transacciones e índices.
Migraciones	Flyway	Versionado reproducible del esquema.
Web	React + TypeScript	Interfaz web tipada y consumo de API.
Móvil	React Native + TypeScript	Cliente móvil tipado sobre la misma API.
Contenedores	Docker / Docker Compose	Entorno reproducible y servicios locales.
Versionado	Git + GitHub	Historial, colaboración, revisión e integración.
CI	GitHub Actions o equivalente definido	Compilación y pruebas automáticas.
IA	Spring AI	Integración responsable de una capacidad basada en modelo generativo.

8. Ruta de aprendizaje en 40 clases

Bloque	Clases	Núcleo de aprendizaje	Evidencia de cierre
1. Ingeniería del entorno y datos	Clases 1–5	Equipo, proyecto asignado, Git/GitHub, Docker, comprensión del problema, entidades, relaciones, PK/FK y primer modelo relacional.	Proyecto comprendido + repositorio operativo + modelo inicial.
2. PostgreSQL y SQL	Clases 6–10	DDL, DML, consultas, joins, agregaciones, constraints, transacciones, índices básicos y migraciones.	Base de datos funcional y versionada; consultas defendibles.
3. Java, Spring y arquitectura	Clases 11–15	Java 21 desde la base conocida en C++, Maven/Gradle, Spring Boot, IoC/DI, HTTP, REST y arquitectura hexagonal.	Primeros casos de uso y endpoints organizados.
4. Persistencia, negocio y pruebas	Clases 16–20	JPA/Hibernate, mapeo, validación, errores, servicios/casos de uso, pruebas unitarias/integración y refactor.	Backend de alcance parcial estable y probado.

Bloque	Clases	Núcleo de aprendizaje	Evidencia de cierre
5. React + TypeScript	Clases 21–25	Fundamentos TypeScript, componentes, props, estado, efectos, formularios, rutas y consumo de API.	Aplicación web consumiendo backend real.
6. Integración y entrega	Clases 26–30	Integración end-to-end, seguridad básica si corresponde, Docker Compose, CI, logging, depuración y estabilización.	Sistema web integrado y reproducible.
7. React Native + TypeScript	Clases 31–35	Fundamentos móviles, navegación, formularios, consumo de API, manejo de estados y pruebas de flujo.	Aplicación móvil con capacidades seleccionadas.
8. Spring AI, hardening y defensa	Clases 36–40	Spring AI, diseño de prompts/contratos, fallbacks, validación, pruebas, documentación, demo y defensa.	Producto final integrado, documentado y defendible.

9. Fundamentos teórico-prácticos que todo estudiante debe dominar

Área	Conceptos que debe explicar	Evidencia en el proyecto
Ingeniería de software	Problema, alcance, actores, requisitos, historias, criterios de aceptación, MVP, deuda técnica.	Backlog y decisiones del proyecto.
Modelado relacional	Entidad, atributo, PK/FK, cardinalidad, optionalidad, normalización, integridad.	DER/modelo relacional + justificación.
SQL/PostgreSQL	DDL, DML, SELECT, JOIN, GROUP BY, transacción, índice, constraint.	Migraciones y consultas ejecutables.
Java 21	Tipos, clases, interfaces, excepciones, colecciones, records, lambdas esenciales.	Código backend comprensible y tipado.
Spring Boot	IoC/DI, configuración, controllers, validation, services, persistence.	Casos de uso expuestos por REST.
HTTP/REST	Métodos, recursos, códigos de estado, JSON, idempotencia, errores.	Contrato de API coherente.
Arquitectura	Módulo, dominio, aplicación, infraestructura, puerto, adaptador, dependencia.	Estructura justificable del backend.
Persistencia ORM	Entity mapping, repository, transacción, lazy/eager, N+1.	Persistencia sin ocultar fundamentos SQL.
Pruebas	Unitarias, integración, fixture, aserción, happy path, errores.	Pruebas relevantes que detecten regresiones.

Área	Conceptos que debe explicar	Evidencia en el proyecto
TypeScript	Tipos, interfaces, unions, narrowing, generics básicos, nullability, async.	Contratos y estado tipados sin abuso de any.
React	Componentes, props, estado, hooks, efectos, formularios, routing.	Web con loading/error/empty states.
React Native	Pantallas, navegación, componentes nativos, estado y API.	Móvil conectado a backend real.
Git/GitHub	Commit, branch, merge, PR, review, conflict, issue.	Historial verificable de colaboración.
Docker/CI	Imagen, contenedor, Dockerfile, Compose, pipeline, build, test.	Entorno reproducible + CI verde.
Spring AI	Modelo, prompt, salida no determinista, timeout, fallback, validación.	Feature de IA útil y degradable.

10. Sistema formal de evaluación

La asignatura contempla dos exámenes parciales, trabajos prácticos y un examen final. Cada proyecto tendrá definido por el docente el alcance exacto que corresponde a cada una de estas instancias. La evaluación no se separa del proyecto: los contenidos teóricos, las decisiones de diseño, la implementación y la capacidad de explicación se verificarán sobre el avance real que corresponda al equipo en ese momento.

La calificación de cada estudiante será individual aunque el producto sea grupal. Para cada instancia evaluativa, el 20% de la nota corresponderá al trabajo de equipo evidenciado mediante el producto y su documentación; el 80% corresponderá a la defensa individual. Por tanto, compartir una nota grupal por haber entregado software no sustituye la obligación de demostrar aprendizaje personal.

Componente de cada evaluación	Peso	Qué acredita	Cómo se verifica
Producto + documentación del equipo	20%	Que el equipo integró y documentó correctamente el avance exigido.	Repositorio, documentación, modelo, código, pruebas, CI, demo y artefactos definidos para el corte.
Defensa individual	80%	Que el estudiante comprende el avance completo exigible del proyecto.	Preguntas, explicación de decisiones, navegación por código/BD, diagnóstico, modificación o demostración técnica individual.

Fórmula por instancia

NOTA INDIVIDUAL DE LA INSTANCIA = (CALIDAD DEL PRODUCTO Y DOCUMENTACIÓN GRUPAL × 0,20) + (DEFENSA INDIVIDUAL × 0,80). La ponderación global entre Trabajos Prácticos, Parcial 1, Parcial 2 y Examen Final se sujetará al esquema académico oficial comunicado por el docente; esta guía fija la composición interna 20/80 de cada nota evaluativa.

10.1 Qué significa el 20% grupal

El componente grupal mide la calidad del incremento que el equipo debía alcanzar en la fecha definida. No se limita a “tener código”. El docente puede considerar integridad del repositorio, cumplimiento del alcance, funcionamiento, organización del software, calidad de la base de datos, pruebas, documentación, trazabilidad de decisiones, evidencias de colaboración y capacidad de ejecutar la solución de forma reproducible.

Una documentación elegante no puede compensar un producto que no funciona, y un producto que “parece funcionar” no puede compensar la ausencia de evidencia técnica. La documentación debe explicar el sistema real entregado, no un diseño ideal que el código todavía no implementa.

10.2 Qué significa el 80% individual

La defensa individual comprueba que el estudiante aprendió. El docente podrá seleccionar cualquier elemento que esté dentro de la frontera evaluable del proyecto y pedir al estudiante que lo explique, lo relacione con otros componentes, identifique errores, ejecute una consulta, siga una petición end-to-end, justifique una decisión o realice una modificación acotada. La defensa puede combinar preguntas conceptuales y demostraciones sobre el proyecto.

El estudiante no puede limitar su preparación a las tareas que personalmente implementó. La división interna del trabajo sirve para organizar la construcción, pero la evaluación individual exige comprensión transversal. Un estudiante que realizó principalmente frontend debe poder explicar el modelo de datos y el endpoint que consume; un estudiante que trabajó en backend debe comprender la pantalla que dispara su caso de uso y el comportamiento visible para el usuario.

10.3 Regla de frontera evaluable

Antes de cada corte, el docente establecerá qué parte del proyecto pertenece a Trabajos Prácticos, Parcial 1, Parcial 2 o Examen Final. Esa frontera puede definirse mediante funcionalidades, módulos, historias de usuario, artefactos y conceptos técnicos. Dentro de esa frontera, el estudiante debe dominar tanto teoría como práctica. Fuera de ella, el trabajo futuro puede existir, pero no constituye el núcleo de la evaluación de ese corte.

Instancia	Foco típico de avance	Ejemplos de evidencia y defensa
Trabajos Prácticos	Prácticas pequeñas e incrementos del proyecto definidos durante el curso.	SQL, migraciones, Git, endpoints, pruebas, componentes TypeScript, integración, ejercicios individuales o grupales.
Parcial 1	Fundamentos y primer incremento integrado, según delimitación del proyecto.	Problema, modelo relacional, PostgreSQL/SQL, Git/Docker inicial, Java/Spring y arquitectura hasta la frontera anunciada.
Parcial 2	Segundo incremento integrado, según delimitación del proyecto.	Backend consolidado, persistencia, REST, pruebas, React + TypeScript, integración y prácticas DevOps incluidas hasta el corte.
Examen Final	Producto final dentro del alcance aprobado.	Web + móvil + backend + BD + CI/Docker + IA definida + documentación + defensa transversal end-to-end.

10.4 Rúbrica detallada del componente grupal (20%)

Criterio	Peso interno	Indicador
Cumplimiento del alcance	25% del componente grupal	Implementa exactamente lo comprometido para el corte, priorizando flujos completos.
Funcionamiento e integración	20%	El incremento puede ejecutarse y demostrar su comportamiento con datos consistentes.
Calidad técnica	20%	Respeto la arquitectura y criterios técnicos enseñados hasta ese momento.
Documentación	20%	Describe problema, decisiones, modelo, API, ejecución, pruebas y cambios de forma coherente con el código real.
Trabajo colaborativo y trazabilidad	15%	Existe evidencia de tareas, commits, PR/revisiones, integración y responsabilidad compartida.

10.5 Rúbrica detallada de la defensa individual (80%)

Criterio	Peso interno	Indicador
Comprensión conceptual	25% del componente individual	Define y relaciona correctamente los conceptos técnicos presentes en el avance.
Comprensión del proyecto completo	25%	Puede recorrer el flujo, módulos, datos y decisiones aunque no haya implementado personalmente cada parte.
Capacidad práctica	20%	Puede ejecutar, consultar, localizar, modificar o demostrar elementos concretos sin depender de otro integrante.
Justificación de decisiones	15%	Explica por qué se eligió una solución y reconoce alternativas, límites y consecuencias.
Diagnóstico y razonamiento	15%	Interpreta errores, propone causas, pruebas y correcciones razonables en lugar de adivinar.

Consecuencia académica importante

Dos integrantes del mismo equipo pueden obtener notas muy diferentes en la misma instancia. El 20% grupal puede ser idéntico para ambos, pero el 80% individual dependerá de la profundidad real de la defensa de cada estudiante.

11. Cómo prepararse para una defensa individual

1. Actualice su repositorio local y ejecute la versión exacta que será evaluada.
2. Revise el alcance oficial del corte y marque cada requisito como comprendido, demostrable y explicable.
3. Dibuje el modelo de datos sin copiarlo y explique PK, FK, cardinalidades y constraints importantes.
4. Elija al menos dos flujos y sígalos de extremo a extremo: interfaz → API → caso de uso → persistencia → respuesta.
5. Revise los módulos y explique qué pertenece a dominio, aplicación e infraestructura.
6. Ejecute consultas SQL relevantes y explique qué devuelve cada JOIN, filtro o agregación.
7. Identifique validaciones y explique dónde se aplican y qué error producen.
8. Revise pruebas existentes: qué comportamiento protegen, qué datos preparan y qué regresión detectarían.
9. En web/móvil, identifique estados de carga, error, vacío y éxito.
10. Revise Docker, variables de entorno y CI para poder explicar cómo se reproduce la solución.
11. Explique al menos una decisión técnica que usted no haya tomado personalmente.
12. Practique diagnosticar un fallo: no memorice únicamente el camino feliz.

12. Preguntas que pueden aparecer en una defensa

- ¿Qué problema resuelve este módulo y qué queda fuera de su responsabilidad?
- ¿Por qué esta relación es uno-a-muchos y dónde se almacena la clave foránea?
- ¿Qué constraint impide que la base de datos llegue a un estado inválido?
- ¿Qué diferencia existe entre una PK, una FK y una restricción UNIQUE?
- ¿Qué consulta SQL usaría para obtener este dato y por qué?
- ¿Qué cambia si esta operación se ejecuta fuera de una transacción?
- ¿Qué sucede desde que el usuario pulsa Guardar hasta que PostgreSQL confirma la operación?
- ¿Por qué este código pertenece al dominio, aplicación o infraestructura?
- ¿Qué dependencia evita un puerto en esta parte de la arquitectura?
- ¿Qué diferencia existe entre entidad de dominio, entidad JPA y DTO en su implementación?
- ¿Por qué este endpoint usa POST/PUT/PATCH/DELETE y qué código HTTP devuelve?
- ¿Cómo se comunica un error de validación al cliente?
- ¿Qué prueba automatizada detectaría una regresión en este caso?
- ¿Qué problema produciría una consulta N+1 en esta funcionalidad?
- ¿Qué tipo TypeScript representa este objeto y por qué no debería ser any?
- ¿Qué diferencia existe entre props y estado en este componente React?
- ¿Por qué este efecto se ejecuta y qué problema puede causar una dependencia incorrecta?
- ¿Cómo maneja la pantalla loading, error, vacío y éxito?
- ¿Cómo llega la aplicación móvil al backend durante desarrollo?
- ¿Qué contiene esta imagen Docker y qué contiene este contenedor?
- ¿Qué verifica el pipeline de CI antes de integrar un cambio?
- ¿Qué ocurre si el proveedor de IA devuelve texto inválido o no responde?
- ¿Qué dato no debería enviarse al modelo de IA?
- ¿Qué commit o pull request revisó usted y qué observación realizó?
- Si este compañero no estuviera presente, ¿podría usted ejecutar y explicar esta funcionalidad?

13. Documentación del proyecto

La documentación debe crecer con el producto. No se recomienda escribir un documento final desde cero la última semana porque se pierde trazabilidad y se describen decisiones que ya no coinciden con la

implementación. Cada corte debe actualizar lo que cambió y conservar una visión coherente del estado actual del sistema.

Artefacto	Contenido mínimo esperado
Descripción del problema	Contexto, usuarios/actores, necesidad, alcance y exclusiones.
Backlog/alcance	Historias/casos y criterios de aceptación del corte.
Modelo de datos	Entidades/tablas, atributos, PK/FK, cardinalidades, restricciones y decisiones relevantes.
Arquitectura	Módulos, capas/zonas, dependencias, puertos/adaptadores y flujo end-to-end.
API	Endpoints relevantes, request/response, estados HTTP y errores.
Web y móvil	Pantallas, navegación, estados y funcionalidades incluidas.
Pruebas	Qué se prueba, estrategia, casos importantes y evidencia de ejecución.
DevOps	Cómo levantar el entorno, Docker, variables, CI y comandos esenciales.
IA	Propósito, entrada/salida, límites, validación, fallback y consideraciones de privacidad.
Bitácora de decisiones	Decisiones relevantes, alternativas, problemas y deuda técnica conocida.

14. Reglas del repositorio y GitHub

- El repositorio oficial es la fuente de verdad del proyecto.
- No se aceptará una “versión final” diferente al repositorio mediante ZIP, memoria USB o mensajería como sustitución del flujo acordado.
- Cada cambio significativo debe poder rastrearse a una tarea, issue o criterio de aceptación.
- Los commits deben ser descriptivos y suficientemente pequeños para comprender la evolución.
- No se deben almacenar contraseñas, tokens, archivos .env reales ni secretos en Git.
- Las integraciones importantes deben revisarse por otro integrante.
- La rama estable definida por el curso debe mantenerse demostrable.
- Los conflictos de merge son responsabilidad del equipo y forman parte del aprendizaje colaborativo.

15. Uso de inteligencia artificial generativa

La IA generativa puede utilizarse como herramienta de apoyo cuando el docente lo permita, pero nunca reemplaza la comprensión. El estudiante es responsable de toda línea de código, consulta, configuración o explicación que incorpore al proyecto, sin importar si fue escrita manualmente, sugerida por un asistente o adaptada desde documentación externa. Durante una defensa, “lo generó la IA” no constituye una explicación técnica.

Spring AI será estudiado además como parte del contenido. Esto exige distinguir el uso personal de una herramienta de IA del diseño de una funcionalidad de software que consume un modelo. En el segundo caso deben considerarse contratos, entradas, salidas, latencia, fallos, respuestas inválidas, privacidad, límites y

mecanismos de degradación. Un modelo generativo no debe sustituir reglas deterministas de negocio que el sistema puede validar directamente.

Regla de responsabilidad

Puede apoyarse en IA para aprender, explorar, depurar o proponer alternativas si la política del docente lo permite. Pero sólo incorpore algo al proyecto cuando pueda explicarlo, ejecutarlo, probarlo y defender por qué es correcto para su caso.

16. Qué no hacer

- Dividir el equipo permanentemente en “el de base de datos”, “el de backend” y “el de frontend”.
- Copiar un proyecto completo de otra fuente y tratar de memorizarlo antes de la defensa.
- Construir muchas funcionalidades incompletas en lugar de pocos flujos integrados y estables.
- Usar JPA sin comprender las tablas, claves y SQL que existen debajo.
- Usar any de forma sistemática para evitar aprender TypeScript.
- Poner reglas de negocio directamente en controllers o componentes visuales por comodidad.
- Mantener código que nadie salvo su autor puede explicar.
- Dejar Docker, pruebas, documentación o integración para los últimos días.
- Considerar que una demo feliz reemplaza pruebas y manejo de errores.
- Enviar datos sensibles innecesarios a un modelo de IA.
- Confundir arquitectura hexagonal con crear interfaces y carpetas sin una razón de dependencia real.

17. Definition of Done de una funcionalidad

- El criterio de aceptación está cumplido.
- El código compila y se integra con la versión estable del equipo.
- Las validaciones y errores relevantes están tratados.
- Las modificaciones de base de datos están versionadas mediante migraciones cuando corresponda.
- Existen pruebas apropiadas o una razón técnica explícita de por qué no corresponden.
- La web o el móvil manejan carga, error y ausencia de datos cuando aplique.
- Otro integrante revisó el cambio importante.
- El pipeline se encuentra verde.
- La documentación mínima se actualizó si cambió una API, decisión, modelo o forma de ejecución.
- La funcionalidad puede demostrarse desde la versión integrada.
- Al menos dos integrantes diferentes pueden explicar el flujo completo de la funcionalidad antes de considerarla terminada.

18. Checklist personal antes de cada corte

- ☐ Conozco el alcance exacto que será evaluado.
- ☐ Puedo levantar el proyecto sin depender de otro compañero.
- ☐ Puedo explicar el modelo de datos correspondiente al corte.
- ☐ Puedo ejecutar y explicar las consultas importantes.
- ☐ Comprendo los casos de uso y endpoints incluidos.
- ☐ Puedo ubicar dominio, aplicación e infraestructura en el código.
- ☐ Puedo seguir por lo menos dos flujos end-to-end.
- ☐ Conozco las validaciones y errores principales.

- ☐ Puedo explicar las pruebas existentes.
- ☐ Comprendo las pantallas web/móvil incluidas hasta el corte.
- ☐ Puedo explicar la configuración de Docker/CI incluida hasta el corte.
- ☐ Conozco los principales problemas técnicos que tuvo el equipo y cómo fueron resueltos.
- ☐ Revisé código realizado por compañeros.
- ☐ Puedo justificar decisiones que no tomé personalmente.
- ☐ La documentación coincide con el software real.
- ☐ No existen secretos o credenciales reales versionados.

19. Compromiso académico esperado

Programación Aplicada exige continuidad. Las tecnologías se apoyan unas sobre otras: una brecha en modelado relacional reaparece cuando se estudia JPA; una brecha en HTTP reaparece al conectar React; una mala comprensión del contrato vuelve a aparecer en móvil; una arquitectura confusa vuelve costosa la integración de pruebas e IA. Por ello, el estudiante debe corregir las brechas cuando aparecen y no acumularlas hasta el examen.

El producto final es importante, pero la evidencia central de aprendizaje será la capacidad individual de explicar cómo está construido, por qué está construido así y qué ocurriría ante cambios o fallos. La meta de la materia es que el estudiante salga con un modelo mental integrado para construir software moderno de manera responsable y pueda profundizar posteriormente en bases de datos, backend, frontend, móvil, DevOps, arquitectura o IA.

20. Glosario técnico integral

Este glosario reúne los términos que aparecerán repetidamente en clases, prácticas, documentación, código y defensas. No debe estudiarse como una lista desconectada de definiciones. Cada término debe relacionarse con una evidencia concreta del proyecto: una tabla, una consulta, una clase, un endpoint, una pantalla, una prueba, una decisión de arquitectura, un commit, un contenedor o una interacción con IA.

Para preparar una defensa, el estudiante debería poder dar una definición propia, señalar dónde aparece el concepto en su solución y explicar una consecuencia práctica de aplicarlo bien o mal. El segundo párrafo de cada entrada enfatiza precisamente esa conexión con el proyecto, porque en Ingeniería de Software el vocabulario es útil cuando permite razonar, comunicar decisiones y diagnosticar sistemas reales.

ACID

Conjunto de propiedades de las transacciones: atomicidad, consistencia, aislamiento y durabilidad.

Describen garantías que una base de datos busca ofrecer cuando varias operaciones deben comportarse como una unidad confiable.

En el proyecto importa cuando un caso de uso modifica varios datos relacionados. El estudiante debe poder explicar qué estado inválido podría quedar si una operación se interrumpe a mitad y por qué una transacción reduce ese riesgo.

Adaptador

Componente que conecta el núcleo de la aplicación con una tecnología o interfaz externa. En arquitectura hexagonal traduce entre el lenguaje del exterior y los puertos definidos por la aplicación.

Un controller REST, un repositorio JPA o un cliente de IA pueden ser adaptadores. Su valor está en evitar que reglas centrales dependan directamente de Spring MVC, PostgreSQL o un proveedor específico.

API

Interfaz de programación que define cómo un componente ofrece operaciones a otros componentes. En este curso, normalmente se materializa como una API HTTP/REST que web y móvil consumen.

Una API no es sólo una lista de URLs. Debe tener contratos, validaciones, respuestas, errores y semántica coherente para que distintos clientes puedan integrarse sin conocer los detalles internos del backend.

Arquitectura hexagonal

Estilo que organiza el software alrededor del núcleo de negocio y utiliza puertos y adaptadores para relacionarlo con tecnologías externas. Busca controlar la dirección de dependencias.

En el proyecto se utilizará de manera simplificada. Lo importante no es dibujar un hexágono, sino poder cambiar o probar adaptadores sin contaminar el dominio con detalles de REST, JPA o infraestructura.

Atomicidad

Propiedad transaccional que establece que una operación compuesta se completa totalmente o no deja efectos parciales.

Si registrar una operación requiere insertar una cabecera y varios detalles, la atomicidad evita que se confirme sólo una parte. El estudiante debe identificar casos donde una transacción sea realmente necesaria.

Atributo

Propiedad que describe una entidad o concepto. En una tabla suele representarse mediante una columna.

No todo dato merece ser una columna independiente y no todo atributo acepta cualquier valor. Tipo, nulabilidad, restricciones y significado deben derivarse del dominio, no de preferencias arbitrarias.

Autenticación

Proceso mediante el cual un sistema verifica quién es un usuario o cliente.

Si se incorpora al proyecto, debe distinguirse de autorización. Autenticarse responde “quién eres”; autorizar responde “qué puedes hacer”. No debe consumir el tiempo central del curso si no está en el alcance.

Autorización

Proceso que decide si una identidad autenticada puede realizar una acción o acceder a un recurso.

Una aplicación puede saber quién es el usuario y aun así negar una operación. El estudiante debe comprender dónde se valida el permiso y evitar basar seguridad únicamente en esconder botones en el frontend.

Backend

Parte del sistema que ejecuta reglas de negocio, coordina casos de uso, accede a datos y expone servicios a clientes.

En esta asignatura el backend se implementa con Java 21 y Spring Boot. Debe ser explicable por módulos y no convertirse en un conjunto de controllers que hablan directamente con la base de datos.

Branch

Línea de desarrollo independiente dentro de Git. Permite trabajar en cambios sin modificar inmediatamente la rama estable.

Las ramas cortas facilitan colaboración y revisión. Deben integrarse con frecuencia; una rama enorme mantenida durante semanas incrementa conflictos y dificulta entender el cambio.

Build

Proceso que transforma el código fuente y sus dependencias en artefactos ejecutables o verificables.

Un build reproducible permite que el proyecto compile de la misma forma en distintos equipos y en CI. Si sólo funciona en la computadora de un integrante, el proceso de construcción todavía no está controlado.

CI - Integración Continua

Práctica en la que cada cambio se integra frecuentemente y se valida automáticamente mediante construcción, pruebas u otras verificaciones.

El pipeline del curso debe actuar como una barrera de calidad mínima. Un CI verde no garantiza software perfecto, pero reduce la probabilidad de integrar cambios que ni siquiera compilan o rompen pruebas conocidas.

Cardinalidad

Cantidad de instancias de una entidad que pueden relacionarse con otra: uno a uno, uno a muchos o muchos a muchos.

La cardinalidad se traduce en claves foráneas o tablas intermedias. Una cardinalidad incorrecta genera datos imposibles o limita situaciones legítimas del problema.

Caso de uso

Acción con significado para un actor o para el negocio que produce un resultado observable.

Un buen caso de uso expresa intención, por ejemplo registrar una reserva o cerrar una solicitud. En arquitectura, ayuda a evitar servicios genéricos sin significado y organiza pruebas alrededor de comportamiento.

Clave foránea - FK

Columna o conjunto de columnas que referencia una clave candidata de otra tabla y materializa una relación entre datos.

Una FK protege integridad referencial. El estudiante debe poder señalar en qué tabla se ubica, qué cardinalidad representa y qué debería ocurrir al eliminar o modificar la fila referenciada.

Clave primaria - PK

Identificador único elegido para distinguir inequívocamente cada fila de una tabla.

Una PK debe ser estable y no nula. En muchos proyectos se utiliza un identificador técnico, pero eso no elimina la necesidad de restricciones UNIQUE para claves naturales relevantes del negocio.

Cliente

Aplicación o componente que consume un servicio ofrecido por otro componente.

React web y React Native móvil son clientes del backend. Ambos deben depender del contrato de la API y no de detalles internos de cómo el servidor guarda o calcula los datos.

Código HTTP

Número estandarizado que comunica el resultado de una interacción HTTP, como 200, 201, 400, 404 o 500.

Elegir códigos coherentes ayuda a que los clientes reaccionen correctamente. Devolver 200 para todo oculta semántica; devolver 500 para un dato inválido confunde un error del cliente con un fallo interno.

Commit

Registro inmutable de un conjunto de cambios en Git junto con autor, fecha y mensaje.

Los commits son evidencia de evolución y colaboración. Un buen commit tiene propósito claro y tamaño razonable; un único commit llamado “final” elimina gran parte de la trazabilidad del proceso.

Componente React

Unidad reutilizable de interfaz que recibe datos, mantiene estado cuando corresponde y produce una representación visual.

Los componentes deben dividirse por responsabilidad y experiencia de usuario, no sólo por tamaño. El estudiante debe explicar qué datos recibe por props y qué estado controla localmente.

Constraint

Regla declarada en la base de datos que restringe los valores o relaciones permitidos.

NOT NULL, UNIQUE, CHECK, PK y FK son ejemplos. Las constraints son una última línea de defensa de la integridad y no deben sustituirse únicamente por validaciones en la interfaz.

Contenedor

Instancia ejecutable creada a partir de una imagen de contenedor. Aísla procesos y dependencias en un entorno reproducible.

En Docker, la imagen es la plantilla y el contenedor es la ejecución. El estudiante debe comprender esta diferencia para diagnosticar persistencia, puertos, redes y reinicios.

Controller

Adaptador de entrada que recibe solicitudes externas, interpreta datos de transporte y delega la operación a la aplicación.

Un controller no debería contener reglas de negocio complejas. Su papel es traducir HTTP hacia un caso de uso y traducir el resultado o error hacia una respuesta HTTP coherente.

CORS

Política del navegador que controla solicitudes entre orígenes diferentes.

Durante el desarrollo React puede ejecutarse en un origen distinto al backend. Configurar CORS correctamente resuelve esa interacción; deshabilitar controles indiscriminadamente no es una estrategia de seguridad.

CRUD

Conjunto básico de operaciones Create, Read, Update y Delete sobre información persistente.

CRUD es útil pero no describe por sí solo un dominio. Un proyecto debe expresar capacidades de negocio y reglas, no convertirse únicamente en formularios genéricos para todas las tablas.

DTO

Objeto diseñado para transportar datos entre límites o capas sin exponer necesariamente el modelo interno completo.

Un DTO puede representar una request o response y contener exactamente los campos del contrato. Separarlo de entidades JPA reduce acoplamiento entre API y persistencia.

DDL

Subconjunto de SQL utilizado para definir estructuras, por ejemplo CREATE, ALTER y DROP.

Las migraciones contienen DDL versionado para que cualquier integrante pueda reproducir el esquema. Cambiar tablas manualmente sin registrar el cambio rompe esa reproducibilidad.

DML

Operaciones SQL que manipulan datos, como INSERT, UPDATE y DELETE; SELECT suele estudiarse junto a ellas aunque algunas clasificaciones lo separan como DQL.

Comprender DML permite razonar sobre qué filas cambian y qué transacción requiere una operación. Un ORM genera SQL, pero no elimina la necesidad de entender sus efectos.

Docker Compose

Herramienta declarativa para definir y ejecutar varios contenedores relacionados mediante un archivo de configuración.

En el curso puede coordinar PostgreSQL y backend, además de redes, volúmenes y variables. La meta es que otro estudiante pueda levantar el entorno de forma predecible.

Dockerfile

Archivo con instrucciones para construir una imagen Docker.

Debe describir cómo empaquetar la aplicación sin depender de pasos manuales ocultos. Un Dockerfile claro forma parte de la reproducibilidad del proyecto.

Dominio

Conjunto de conceptos, reglas y lenguaje del problema que el software busca resolver.

El dominio no es equivalente a “todas las clases”. En arquitectura hexagonal, las reglas esenciales deberían permanecer lo más independientes posible de detalles técnicos.

Endpoint

Combinación accesible de método y ruta mediante la cual una API ofrece una operación.

GET /reservas y POST /reservas son endpoints distintos aunque compartan ruta. Cada endpoint debe tener intención, contrato, estados y errores entendibles.

Entidad

Concepto con identidad propia que mantiene continuidad a lo largo del tiempo. En base de datos suele corresponder a una tabla relevante, aunque dominio y persistencia no siempre son idénticos.

Identificar entidades exige comprender el problema. Crear una tabla por cada sustantivo o una clase por cada pantalla suele producir modelos débiles.

Entidad JPA

Clase Java mapeada por JPA/Hibernate a una estructura persistente.

Es un detalle de persistencia. Puede coincidir parcialmente con una entidad de dominio, pero el estudiante debe entender que la anotación @Entity introduce una preocupación técnica concreta.

Estado

Información mutable que determina el comportamiento o representación actual de un sistema o componente.

En React, cambios de estado disparan nuevas representaciones. En dominio, estados como PENDIENTE o CERRADO pueden tener transiciones y reglas que deben validarse explícitamente.

Excepción

Mecanismo para representar y propagar condiciones anómalas durante la ejecución.

No todas las excepciones significan error 500. Las excepciones de negocio o validación deben transformarse adecuadamente en respuestas que expresen el problema real.

Fallback

Comportamiento alternativo que se ejecuta cuando una dependencia o capacidad principal falla o no está disponible.

En una feature con IA, un fallback puede devolver una respuesta controlada o permitir continuar sin la recomendación. Evita que una función probabilística rompa el flujo transaccional principal.

Flyway

Herramienta de migraciones que aplica scripts versionados de esquema en un orden controlado.

Permite que la base de datos evolucione junto con el repositorio. Cada cambio estructural relevante debe ser reproducible y no depender de modificaciones manuales no documentadas.

Foreign key

Término inglés para clave foránea. Define integridad referencial entre tablas.

Debe poder explicarse en ambos vocabularios porque documentación técnica y mensajes de herramientas usan frecuentemente la forma inglesa FK/foreign key.

Frontend

Parte del sistema con la que interactúa directamente el usuario.

Web y móvil son frontends diferentes que consumen el mismo backend. Deben gestionar estados de interfaz y errores, no duplicar reglas de negocio que pertenecen al servidor.

Generic - TypeScript

Mecanismo para definir funciones o tipos parametrizados que conservan información de tipos sin fijar uno único.

En el curso se utilizarán generics básicos cuando aporten claridad, por ejemplo respuestas paginadas o utilidades. No se busca metaprogramación de tipos compleja.

Git

Sistema distribuido de control de versiones que registra la historia del código y permite trabajar en ramas.

Git es una herramienta de ingeniería, no sólo un mecanismo para entregar. El historial permite integrar, comparar, revisar y recuperar cambios durante la evolución del proyecto.

GitHub

Plataforma que aloja repositorios Git y añade colaboración mediante issues, pull requests, revisiones y automatizaciones.

En la materia constituye parte de la evidencia de trabajo en equipo. No basta con subir archivos al final; debe reflejar el proceso real de construcción.

GROUP BY

Cláusula SQL que agrupa filas para calcular agregaciones por uno o más criterios.

Permite responder preguntas como cantidad de reservas por estado o total por categoría. El estudiante debe distinguir filtrado de filas con WHERE y filtrado de grupos con HAVING.

Hook

Función de React que permite utilizar capacidades como estado, efectos o contexto dentro de componentes funcionales.

Hooks como useState y useEffect son centrales, pero deben entenderse por comportamiento. Copiar un efecto sin comprender dependencias puede producir llamadas repetidas o datos desactualizados.

HTTP

Protocolo de comunicación basado en solicitudes y respuestas utilizado ampliamente en aplicaciones web.

El estudiante debe comprender método, URL, headers, body y status. REST se construye sobre HTTP; ignorar su semántica suele producir APIs difíciles de consumir.

Idempotencia

Propiedad por la cual repetir una operación produce el mismo efecto observable que ejecutarla una vez, dentro de ciertas condiciones.

GET debería ser seguro e idempotente; PUT suele diseñarse como idempotente. Comprender esto ayuda a tratar reintentos y evita duplicar operaciones sensibles.

Índice

Estructura de base de datos que acelera ciertos accesos a cambio de espacio y costo adicional de mantenimiento.

No toda columna necesita índice. Debe justificarse por patrones de consulta, selectividad y uso; agregar índices sin criterio puede perjudicar escrituras y aumentar complejidad.

Inmutabilidad

Propiedad de un objeto o valor que no cambia después de ser creado.

Los valores inmutables facilitan razonamiento y reducen efectos inesperados. Records de Java y prácticas de estado inmutable en React ayudan a trabajar con este principio.

Integración

Proceso de conectar componentes para verificar que colaboran correctamente como sistema.

Que backend y frontend funcionen por separado no garantiza integración. El proyecto debe probar flujos completos con contratos, datos y errores reales.

Integridad referencial

Garantía de que las referencias entre tablas apuntan a filas válidas según las restricciones declaradas.

Una FK impide, por ejemplo, registrar un detalle asociado a una cabecera inexistente. Es una propiedad de datos, no una simple convención del equipo.

IoC - Inversión de Control

Principio en el que la creación y coordinación de dependencias se delega a un contenedor o framework en lugar de ser controlada manualmente por cada objeto.

Spring aplica IoC al gestionar beans. El estudiante debe comprender que el objetivo es controlar dependencias, no simplemente aprender anotaciones.

Inyección de Dependencias - DI

Técnica mediante la cual un objeto recibe sus dependencias desde el exterior.

Constructor injection hace visibles las dependencias y facilita pruebas. Crear repositorios o clientes directamente dentro de un servicio aumenta acoplamiento y dificulta sustitución.

JPA

Especificación de Java para persistencia de objetos en bases relacionales mediante mapeo objeto-relacional.

JPA define contratos y anotaciones; Hibernate suele ser la implementación. El estudiante debe distinguir ambos y recordar que debajo sigue existiendo SQL y un modelo relacional.

Hibernate

Implementación ORM ampliamente utilizada de JPA. Gestiona mapeo, contexto de persistencia y generación de SQL, entre otras funciones.

Hibernate simplifica operaciones repetitivas, pero puede generar consultas costosas si se usa sin comprensión. El problema N+1 es un ejemplo clásico que el estudiante debe reconocer.

JOIN

Operación SQL que combina filas de tablas relacionadas según una condición.

INNER JOIN y LEFT JOIN expresan necesidades distintas. El estudiante debe poder predecir qué filas se conservan y cómo la cardinalidad afecta el resultado.

JSON

Formato textual de intercambio de datos basado en objetos, arreglos, strings, números, booleanos y null.

Es el formato habitual de requests y responses REST en el curso. Un contrato JSON debe estar alineado con tipos TypeScript y DTOs Java sin exponer accidentalmente campos internos.

JWT

Formato de token firmado usado frecuentemente para transmitir claims de identidad o autorización.

Puede aparecer si el curso incorpora autenticación, pero no es obligatorio. Un JWT no cifra por defecto su contenido y no debe tratarse como un lugar para secretos.

Lazy loading

Estrategia en la que datos relacionados se cargan sólo cuando son accedidos.

En ORM puede reducir trabajo inicial, pero también generar múltiples consultas inesperadas o fallar fuera del contexto de persistencia. Debe entenderse junto con transacciones y N+1.

LLM

Large Language Model: modelo entrenado para procesar y generar lenguaje y otros tipos de secuencias.

En Spring AI se consume como dependencia externa probabilística. No debe tratarse como base de datos ni como una función determinista que siempre devuelve el mismo resultado.

Migración de base de datos

Cambio versionado que modifica de forma reproducible el esquema o ciertos datos estructurales.

Cada migración forma parte de la historia del software. Editar una migración ya aplicada puede romper entornos; normalmente se agrega una nueva evolución en lugar de reescribir el pasado.

Módulo

Unidad funcional del sistema que agrupa responsabilidades relacionadas y define un límite interno.

Un monolito modular usa módulos para evitar una única masa de código. Los módulos deberían tener propósito de negocio y dependencias controladas.

Monolito modular

Aplicación desplegada como una unidad pero organizada internamente en módulos con límites claros.

Es la arquitectura base del curso porque permite practicar diseño y separación sin introducir complejidad distribuida prematura. Puede evolucionar más adelante si existe una razón real.

MVP

Minimum Viable Product: versión mínima que entrega valor y permite validar los flujos principales.

En la materia significa alcance pequeño pero completo. Es preferible tener tres capacidades integradas y defendibles que doce pantallas sin reglas, pruebas ni estabilidad.

N+1

Problema de acceso a datos donde una consulta inicial provoca una consulta adicional por cada elemento recuperado.

Suele aparecer con ORM y relaciones cargadas de forma inadecuada. El estudiante debe aprender a observar SQL/logs y corregir la estrategia, no asumir que JPA siempre genera consultas óptimas.

Normalización

Proceso de estructurar un modelo relacional para reducir redundancia y anomalías, utilizando dependencias y formas normales como guía.

En este curso se busca una aplicación práctica, especialmente hasta 3FN cuando corresponda. La meta no es memorizar reglas aisladas sino justificar por qué un dato está ubicado en una tabla.

NULL

Marcador relacional para indicar ausencia o desconocimiento de valor; no equivale a cero ni a string vacío.

La nulabilidad debe diseñarse conscientemente. En SQL tiene lógica especial y en TypeScript/Java requiere tratamiento para evitar errores y contratos ambiguos.

ORM

Object-Relational Mapping: técnica para mapear objetos del lenguaje a estructuras de una base relacional.

Reduce código repetitivo pero introduce una capa de abstracción. El curso enseña SQL primero para que el estudiante pueda entender qué hace el ORM y diagnosticar problemas.

Pipeline

Secuencia automatizada de pasos que procesa cambios de software, por ejemplo checkout, build, test y empaquetado.

En CI, el pipeline debe ser repetible y visible. Un fallo del pipeline es información técnica que el equipo debe investigar, no un obstáculo que se ignora antes de fusionar.

PostgreSQL

Sistema gestor de bases de datos relacional y objeto-relacional de código abierto, robusto y ampliamente utilizado.

Será la base principal del proyecto. El estudiante debe manejar modelo, SQL, constraints, transacciones, índices básicos y conexión desde Spring.

Primary key

Término inglés para clave primaria. Identifica de forma única una fila.

Aparece frecuentemente en diagramas, documentación y mensajes de herramientas. Debe distinguirse de claves foráneas y claves naturales.

Prompt

Entrada de instrucciones y contexto que se entrega a un modelo generativo.

Un prompt de producción debe diseñarse como parte de un contrato: qué recibe, qué se espera, qué restricciones tiene y cómo se valida la salida. No es magia ni reemplaza reglas.

Props

Valores que un componente React recibe desde su padre para configurar su representación o comportamiento.

Las props son una forma de comunicación explícita. Un componente debería declarar tipos claros para ellas y evitar depender de datos globales sin necesidad.

Puerto

Interfaz que define cómo el núcleo de la aplicación puede ser utilizado o qué capacidades externas necesita.

Los puertos permiten expresar dependencias en términos del negocio o aplicación. Un repositorio de dominio no debería depender de JpaRepository si se busca mantener el límite hexagonal.

Pull Request - PR

Solicitud formal para revisar e integrar cambios de una rama a otra en plataformas como GitHub.

Es un punto de conversación técnica. Debe permitir revisar propósito, código, pruebas y riesgos antes de integrar, y deja evidencia de aprendizaje colaborativo.

React

Biblioteca para construir interfaces de usuario mediante componentes y actualización declarativa de la vista.

En el curso se utilizará con TypeScript. El estudiante debe aprender componentes, props, estado, hooks, formularios, rutas y consumo de API sin trasladar reglas de negocio del backend al navegador.

React Native

Framework basado en React para crear interfaces móviles con componentes nativos.

Permite reutilizar el modelo mental de React, pero no es simplemente una web dentro del teléfono.

Navegación, permisos, entorno y componentes tienen particularidades móviles.

Repository

Abstracción usada para acceder y persistir agregados o entidades sin exponer necesariamente detalles de almacenamiento.

En arquitectura hexagonal puede existir como puerto de salida y tener un adaptador JPA. El estudiante debe diferenciar la idea de repositorio de negocio de una interfaz técnica generada por Spring Data.

REST

Estilo arquitectónico para sistemas distribuidos basado en recursos, representaciones e interfaz uniforme, normalmente sobre HTTP.

En el curso se aplicarán sus principios prácticos al diseñar endpoints coherentes. REST no significa únicamente devolver JSON desde un controller.

Rollback

Reversión de una transacción para descartar cambios cuando no puede completarse de forma válida.

Es esencial cuando una operación falla después de haber realizado pasos parciales. El estudiante debe relacionarlo con atomicidad y manejo de excepciones.

Routing

Mecanismo que determina qué vista o componente se muestra para una ruta o destino.

En web se relaciona con URLs; en móvil con navegadores y pantallas. Debe organizar la experiencia sin duplicar reglas de negocio.

Schema

Estructura lógica que organiza objetos de base de datos; también se usa genéricamente para describir la forma de datos.

En PostgreSQL puede separar espacios de nombres. En APIs, “schema” también puede referirse a la estructura esperada de un JSON, por lo que el contexto debe quedar claro.

Servicio de aplicación

Componente que implementa o coordina uno o más casos de uso.

Debe orquestar dominio y puertos sin absorber todos los detalles técnicos. Un servicio gigantesco suele indicar responsabilidades mal divididas.

Spring AI

Proyecto del ecosistema Spring que ofrece abstracciones e integraciones para trabajar con modelos de IA y capacidades relacionadas.

En el curso se utilizará para una funcionalidad concreta y limitada. El estudiante deberá manejar configuración, prompt, errores, validación y fallback, no sólo obtener una respuesta del modelo.

Spring Boot

Framework/plataforma que simplifica la creación y configuración de aplicaciones Spring mediante convenciones, starters y auto-configuración.

Permite concentrarse en casos de uso y arquitectura sin configurar cada componente manualmente. Aun así, el estudiante debe comprender qué dependencias y beans existen realmente.

SQL

Lenguaje declarativo para definir, consultar y modificar datos relacionales.

Es una competencia fundamental del curso. El ORM no sustituye la capacidad de leer y escribir SQL, analizar joins, comprender transacciones y verificar integridad.

Status de interfaz

Representación del estado operativo de una pantalla: carga, éxito, vacío, error, deshabilitado, entre otros.

Una interfaz profesional no asume que toda llamada responde inmediatamente. React y React Native deben mostrar estados coherentes para que el usuario comprenda qué está ocurriendo.

Transacción

Unidad lógica de trabajo cuyas operaciones se confirman o revierten según reglas de consistencia.

Debe delimitarse alrededor de una operación de negocio, no utilizarse como decoración. Una transacción demasiado grande también puede aumentar bloqueos y costo.

Type

Construcción de TypeScript para definir alias y composiciones de tipos.

Puede representar DTOs, unions y estructuras de estado. Elegir type o interface depende del caso; lo importante es que el contrato sea explícito y útil.

Interface - TypeScript

Declaración de la forma que debe cumplir un objeto o contrato de tipos.

Se utiliza para modelar props, entidades de vista o contratos recibidos. Ayuda al editor y compilador a detectar incompatibilidades antes de ejecutar.

TypeScript

Superset tipado de JavaScript que añade análisis estático y herramientas para describir contratos.

Será obligatorio en web y móvil. El objetivo no es convertir todo en tipos complejos, sino detectar errores antes, comunicar intenciones y mantener coherencia entre API y clientes.

Union type

Tipo TypeScript que permite uno de varios tipos o valores posibles.

Es útil para estados discriminados como loading | success | error o roles limitados. Bien usado hace explícitas alternativas válidas y ayuda al narrowing.

Narrowing

Proceso mediante el cual TypeScript reduce un tipo amplio a uno más específico utilizando comprobaciones de control de flujo.

Permite tratar con seguridad unions, valores opcionales y respuestas. En lugar de usar as o any para silenciar errores, el estudiante debe aprender a comprobar qué caso tiene realmente.

any

Tipo de TypeScript que desactiva gran parte de la verificación estática para un valor.

Puede ser útil en casos excepcionales, pero su uso indiscriminado elimina la principal ventaja de TypeScript. En el curso debe justificarse y preferirse unknown o tipos concretos cuando sea posible.

unknown

Tipo de TypeScript que representa un valor desconocido y obliga a verificarlo antes de usarlo.

Es más seguro que any al procesar datos externos, errores o respuestas no confiables. Fuerza al estudiante a validar antes de asumir una estructura.

Nullability

Propiedad de un dato que indica si puede estar ausente o ser null.

Debe mantenerse coherente entre base de datos, Java, JSON y TypeScript. Muchas fallas de integración nacen de contratos que no aclaran cuándo un dato puede faltar.

Validación

Proceso de verificar que una entrada cumple reglas antes de ser aceptada.

Puede existir en interfaz por usabilidad, en backend por seguridad e invariantes y en base de datos por integridad. Cada capa tiene una responsabilidad distinta y complementaria.

Variable de entorno

Valor de configuración proporcionado al proceso sin codificarlo directamente en el repositorio.

Se usa para URLs, credenciales o claves de proveedores. Los secretos reales deben mantenerse fuera de Git y documentarse mediante nombres/plantillas seguras.

Volumen Docker

Mecanismo para persistir o compartir datos fuera del ciclo de vida efímero de un contenedor.

PostgreSQL necesita persistencia para no perder datos cada vez que se recrea el contenedor. El estudiante debe distinguir volumen de imagen y de filesystem interno temporal.

WebHook

Mecanismo mediante el cual un sistema envía una solicitud HTTP a otro cuando ocurre un evento.

No es obligatorio en el curso, pero es útil distinguirlo de polling o consumo normal de API. Si aparece en un proyecto, debe manejar autenticidad, reintentos e idempotencia.

WebSocket

Protocolo que mantiene comunicación bidireccional persistente entre cliente y servidor.

No es un requisito base y debe evitarse si HTTP normal resuelve el caso. Introducir tecnología en tiempo real sólo se justifica cuando el problema realmente la necesita.

Wireframe

Representación esquemática de una pantalla que define estructura y jerarquía antes del diseño visual detallado.

Ayuda a validar flujos y campos antes de programar React o React Native. Cambiar un wireframe cuesta menos que rehacer componentes ya integrados.

Historia de usuario

Forma breve de describir una necesidad desde la perspectiva de un actor, normalmente acompañada por criterios de aceptación.

Sirve para organizar alcance y conversación, no para reemplazar todo análisis. En el proyecto debe poder conectarse con una funcionalidad implementada y una evidencia verificable.

Criterio de aceptación

Condición verificable que define cuándo una historia o requerimiento cumple lo esperado.

Reduce ambigüedad y facilita pruebas y demos. “Gestionar reservas” es vago; criterios concretos indican qué entradas, reglas y resultados deben existir.

Backlog

Lista priorizada de trabajo pendiente o capacidades del producto.

En el curso ayuda a delimitar qué corresponde a cada corte. Un backlog sano cambia con decisiones, pero las fronteras evaluables comunicadas por el docente deben respetarse.

MVP - Producto Mínimo Viable

Versión más pequeña que resuelve de forma coherente un conjunto esencial de necesidades y permite validar el producto.

No significa software desordenado o sin calidad. Significa controlar alcance: pocos flujos completos, con datos, backend, interfaz y pruebas suficientes para ser defendibles.

Regla de negocio

Condición o política derivada del dominio que determina qué operaciones o estados son válidos.

Debe vivir donde pueda protegerse de todos los clientes. Si sólo existe en React, una llamada directa a la API podría omitirla; por ello el backend debe proteger invariantes relevantes.

Invariante

Condición que debe mantenerse verdadera para que un objeto o agregado permanezca en un estado válido. Ejemplos: una cantidad no puede ser negativa o una reserva cerrada no puede volver a borrador. Expresar invariantes hace el modelo más fuerte y las pruebas más significativas.

Agregado

Grupo de objetos de dominio tratados como una unidad de consistencia alrededor de una raíz.

Es un concepto de DDD que puede usarse de forma ligera. No todo proyecto del curso necesitará agregados complejos, pero ayuda a razonar qué cambios deben ser atómicos.

Value Object

Objeto definido por sus valores y no por una identidad propia, normalmente inmutable.

Dinero, rango de fechas o email pueden modelarse así cuando aporta claridad y validación. En un curso inicial debe utilizarse con criterio, evitando abstracción innecesaria.

DDD

Domain-Driven Design: enfoque para diseñar software complejo alrededor del modelo y lenguaje del dominio.

La materia no exige aplicar DDD completo. Se tomarán ideas útiles como lenguaje ubicuo, reglas, entidades, value objects y módulos cuando ayuden a entender el problema.

Lenguaje ubicuo

Vocabulario compartido entre personas del dominio y equipo técnico para describir conceptos sin traducciones ambiguas.

Nombrar tablas, clases, endpoints y documentación con el mismo vocabulario reduce confusión. Un mismo concepto no debería llamarse de tres maneras diferentes en cada capa.

OpenAPI

Especificación para describir APIs HTTP de forma legible por humanos y herramientas.

Puede usarse para documentar endpoints, modelos y respuestas. Si se incorpora, debe reflejar el contrato real y no convertirse en documentación desactualizada.

Logging

Registro estructurado de eventos relevantes de ejecución para diagnóstico y operación.

Los logs deben ayudar a responder qué ocurrió sin exponer secretos. Imprimir todo indiscriminadamente genera ruido y puede filtrar información sensible.

Observabilidad

Capacidad de comprender el estado interno de un sistema a partir de señales como logs, métricas y trazas.

En el curso se aborda en nivel introductorio mediante logging y diagnóstico. La meta es aprender a investigar fallos con evidencia, no únicamente con prueba y error.

Refactorización

Cambio interno de estructura que mejora diseño o mantenibilidad sin alterar el comportamiento observable esperado.

Debe hacerse con pruebas y pequeños pasos. Reescribir grandes secciones justo antes de un corte aumenta riesgo si no existe cobertura ni comprensión compartida.

Deuda técnica

Costo futuro creado por decisiones que aceleran el presente pero dejan limitaciones, fragilidad o trabajo pendiente.

Toda solución real tiene compromisos. Lo importante es reconocer la deuda, documentarla y evitar confundir un atajo temporal con una arquitectura definitiva.

Prueba unitaria

Prueba automatizada que verifica una unidad pequeña de comportamiento de manera rápida y controlada. Es apropiada para reglas y casos de uso que pueden aislarse. No necesita levantar toda la aplicación para cada comportamiento.

Prueba de integración

Prueba que verifica colaboración entre varios componentes reales o casi reales, por ejemplo repositorio y base de datos.

Complementa a las unitarias. Debe elegirse cuando el riesgo está en configuración, mapeo, SQL o interacción entre piezas que una prueba aislada no puede descubrir.

Mock

Objeto de prueba que simula una dependencia y permite controlar interacciones.

Puede facilitar pruebas unitarias, pero abusar de mocks produce pruebas acopladas a implementación. Se debe simular sólo lo necesario para verificar comportamiento.

Fixture

Conjunto de datos o configuración preparada para ejecutar una prueba de forma repetible.

Fixtures claros reducen ambigüedad. Deben ser mínimos para que la prueba exprese qué condición necesita y por qué falla.

Aserción

Comprobación automatizada de que un resultado cumple una condición esperada.

Una prueba sin aserciones significativas puede ejecutar código sin verificar nada. Las aserciones deben expresar el comportamiento que se pretende proteger.

Happy path

Escenario principal en el que entradas y dependencias se comportan como se espera.

Es necesario pero insuficiente. El proyecto también debe considerar datos inválidos, ausencia de registros, conflictos, timeouts y otros caminos relevantes de error.

Timeout

Límite de tiempo tras el cual una operación se considera fallida o se cancela.

Es especialmente importante al llamar servicios externos o IA. Sin timeout, una dependencia lenta puede bloquear recursos y deteriorar toda la experiencia.

Retry

Reintento controlado de una operación después de un fallo potencialmente temporal.

No toda operación debe reintentarse. Si no es idempotente, repetirla puede duplicar efectos; por ello retry e idempotencia deben razonarse juntos.

Serialización

Conversión de un objeto o estructura a un formato transportable o almacenable, como JSON.

Spring serializa DTOs a JSON y los clientes los deserializan a tipos TypeScript. Campos, fechas y nullability deben mantenerse coherentes entre ambos lados.

Deserialización

Reconstrucción de una estructura en memoria a partir de un formato como JSON.

Los datos externos no deben asumirse válidos sólo porque pueden deserializarse. Se requieren validaciones y manejo de errores cuando el contrato no se cumple.

Contrato

Acuerdo explícito sobre entradas, salidas, reglas y comportamiento entre componentes.

Una API, un puerto o un componente tipado tiene contrato. La calidad del sistema aumenta cuando los contratos son claros, probables y no dependen de conocimiento implícito.

Responsabilidad

Razón principal por la que existe un módulo, clase o componente y conjunto de decisiones que le corresponden.

Preguntar “¿de quién es esta responsabilidad?” ayuda a evitar controllers gigantes, servicios todopoderosos y componentes React que mezclan red, negocio y presentación.

Acoplamiento

Grado en que un componente depende de detalles de otro.

Un acoplamiento alto hace costosos los cambios. Los puertos, DTOs y módulos buscan reducir acoplamiento donde aporta valor, no eliminar todas las dependencias posibles.

Cohesión

Grado en que los elementos de un módulo están relacionados por un propósito común.

Una alta cohesión facilita comprensión. Un módulo que mezcla usuarios, reportes, pagos y configuración sin una razón de dominio es difícil de mantener y defender.

Dependency direction

Dirección en la que un módulo conoce a otro y, por tanto, queda afectado por sus cambios.

En arquitectura hexagonal se procura que detalles externos dependan de contratos internos, no que el dominio importe controladores o repositorios técnicos.

Spring Bean

Objeto administrado por el contenedor de Spring.

El contenedor controla su ciclo de vida e inyección. No toda clase debe ser bean; se declaran como beans las piezas cuya creación o colaboración necesita gestionar Spring.

Annotation

Metadato declarado con @ en Java que puede ser interpretado por compilador, frameworks o herramientas.

Spring y JPA usan muchas anotaciones. El estudiante debe comprender su efecto y no tratar @Service, @Entity o @Transactional como palabras mágicas sin semántica.

Record - Java

Tipo conciso para representar datos inmutables con componentes definidos en Java.

Puede ser apropiado para DTOs o valores simples. No reemplaza automáticamente entidades de dominio que requieren identidad, comportamiento o ciclo de vida.

Optional - Java

Contenedor que representa explícitamente presencia o ausencia de un valor.

Es útil en retornos donde el dato puede no existir, por ejemplo búsquedas. No debe utilizarse indiscriminadamente en todos los campos o entidades.

Lambda

Expresión concisa para representar comportamiento como función en Java y JavaScript/TypeScript.

En Java aparece con streams y callbacks; en React es común en manejadores. La concisión no debe ocultar lógica compleja que merece un nombre claro.

Stream - Java

API para procesar secuencias de elementos mediante operaciones declarativas como map, filter y reduce. Permite transformar colecciones con claridad cuando se usa con moderación. Una cadena excesiva puede ser más difícil de depurar que un bucle sencillo.

Promise

Abstracción JavaScript/TypeScript que representa el resultado futuro de una operación asíncrona.

Las llamadas HTTP en React y React Native devuelven promesas. El estudiante debe manejar resolución y error, normalmente mediante async/await y try/catch.

async/await

Sintaxis para escribir flujo asíncrono basado en promesas de forma más secuencial y legible.

No convierte una llamada remota en síncrona ni elimina fallos. Debe combinarse con estados de carga, manejo de errores y cancelación cuando corresponda.

useState

Hook de React para mantener estado local de un componente.

Debe usarse para información que cambia y afecta la UI. Guardar todo en estado, incluso valores derivados, puede generar inconsistencias innecesarias.

useEffect

Hook de React para sincronizar un componente con sistemas externos o realizar efectos después del render.

Es frecuente para cargar datos, pero no todo cálculo necesita un efecto. Sus dependencias deben comprenderse para evitar ciclos y datos obsoletos.

Responsive design

Diseño de interfaz que se adapta a diferentes tamaños y características de pantalla.

En web debe considerar escritorio y tamaños reducidos. React Native exige además pensar desde componentes y patrones propios del dispositivo móvil.

UX

User Experience: calidad de la experiencia completa del usuario al lograr sus objetivos con el producto.

No es sólo estética. Estados claros, validaciones útiles, tiempos de respuesta, navegación y mensajes de error forman parte de UX y deben observarse en la demo.

UI

User Interface: elementos visuales e interactivos mediante los cuales el usuario opera el sistema.

Una UI consistente comunica estado y acciones posibles. Debe estar alineada con reglas del sistema, pero no ser el lugar único donde se aplican reglas de negocio.

Endpoint idempotente

Endpoint diseñado para que repetir la misma solicitud no multiplique sus efectos.

Es relevante para PUT y para operaciones que pueden reintentarse. El estudiante debe reconocer cuándo una repetición accidental podría crear duplicados.

Error 400

HTTP Bad Request: solicitud inválida según sintaxis o validación del contrato.

Debe distinguirse de 404, 409 o 500. Una respuesta de error útil incluye suficiente información para que el cliente pueda informar o corregir la acción.

Error 404

HTTP Not Found: el recurso solicitado no existe en el contexto indicado.

No debe usarse como respuesta genérica para cualquier error. Su semántica permite que web y móvil distingan ausencia de un fallo interno.

Error 409

HTTP Conflict: la solicitud es comprensible pero entra en conflicto con el estado actual o una restricción.

Puede representar duplicados, transiciones inválidas u otros conflictos según el contrato. Debe acompañarse de un mensaje o código de error de aplicación consistente.

Error 500

HTTP Internal Server Error: fallo inesperado del servidor.

No debe usarse para errores de negocio previsibles. Además, la respuesta al cliente no debe exponer stack traces, secretos o detalles internos.

21. Cierre: qué significa aprobar con aprendizaje real

Aprobar Programación Aplicada debe significar que el estudiante puede sentarse frente a una aplicación construida en equipo y explicar con seguridad cómo el problema se convirtió en datos, reglas, API, interfaces, pruebas y operación. También debe poder reconocer qué no sabe todavía, localizar información técnica y avanzar sin depender de recetas ciegas.

El proyecto es el medio para demostrar ese aprendizaje. La nota individual se construye sobre la capacidad de dominar el avance compartido, comunicarlo con precisión y actuar técnicamente sobre él. El equipo entrega un producto; cada estudiante demuestra que realmente aprendió a construirlo.